

## Aula 15 - Fundamentos JS e Funções

**Descrição:** Introdução ao desenvolvimento client-side com JavaScript. Abordaremos a declaração de variáveis, escopo, tipos de dados, operadores lógicos e aritméticos, além de construir funções utilitárias reutilizáveis para manipulação e validação de dados em aplicações web.

---

**Cenário Real:** A web moderna é dinâmica. Quando você acessa uma loja virtual e preenche os dados do seu cartão de crédito, o sistema não espera você clicar em "Comprar" e enviar tudo para o servidor (backend) para só então avisar que faltou um número no cartão. Ele faz isso instantaneamente, direto no seu navegador. O responsável por essa magia, economizando tempo do usuário e processamento no servidor, é o JavaScript (JS). Hoje, daremos o primeiro passo para dar vida às nossas interfaces estáticas.

**Pergunta:** Vocês já preencheram um formulário que avisou que a senha estava fraca ou que o e-mail era inválido antes mesmo de vocês clicarem em "Enviar"? Como vocês acham que o navegador consegue analisar essa informação tão rápido sem precisar recarregar a página?

---

### Onde guardamos os dados: Variáveis, Tipos e Escopo

#### A. Conceito:

Em JavaScript, lidamos com diversos **tipos comuns** de dados:

- `String` (textos),
- `Number` (números),
- `Boolean` (verdadeiro/falso),
- além de valores vazios como `null` (representa a ausência intencional de qualquer valor de objeto) e `undefined` (indica que algo existe mas ainda não recebeu nenhum valor).

Para organizar dados complexos, usamos estruturas como:

- `Array` (listas) e
- `Object` (coleções de pares chave-valor).

Para armazenar esses dados na memória, precisamos declará-los usando palavras-chave específicas. O JavaScript moderno nos oferece `let` e `const` para declaração de variáveis, substituindo o antigo `var`. A diferença principal entre eles é o **escopo**, ou seja, onde a variável "existe" e pode ser acessada. Variáveis criadas com `let` e `const` possuem escopo de bloco (existem apenas dentro das chaves `{ }` onde foram criadas).

O `var` possui escopo de função ou global, o que frequentemente causa vazamento de dados e bugs difíceis de rastrear em sistemas maiores. A escolha entre `let` e `const` define a mutabilidade da referência:

- Nós usamos `const` quando o valor (ou a referência do objeto/array) não deve ser reatribuído durante a execução do script.
- Nós usamos `let` quando sabemos que o valor vai mudar, como em um contador dentro de um laço de repetição.

É uma boa prática de segurança e clareza iniciar tudo como `const` e só mudar para `let` se for estritamente necessário.

#### B. Explicação didática:

Imaginem as variáveis como **caixas organizadoras**. O tipo de dado é o que vai dentro: uma `String` é um bilhete de papel, um `Number` é uma calculadora, um `Array` é uma pasta com várias divisórias e um `Object` é uma maleta cheia de ferramentas rotuladas.

O escopo é onde você deixa essa caixa. O `let` e o `const` são caixas que ficam trancadas dentro da sala onde você as colocou (escopo de bloco). Ninguém do corredor consegue mexer. O `const` é um cofre:

depois que você guarda algo lá dentro, não pode trocar por outra coisa. Já o `var` é como deixar a caixa no meio da rua: qualquer um que passar pode abrir, mudar o que tem dentro e bagunçar a sua vida.

### C. Visualizando:

Um diagrama de círculos concêntricos: o círculo maior representa o escopo global. Dentro dele, círculos médios representam escopos de função, e dentro deles, círculos menores representam escopos de bloco (`if`, `for`). Setas mostram que o `var` ignora a barreira do círculo menor e "vaza" para fora, enquanto `let` e `const` ficam contidos e seguros dentro do seu círculo original. {ver img01.png}

### D. Exemplo de Código:

JavaScript

```
// Declarando dados primitivos e estruturais
```

```
const nomeProduto = "Cadeira Gamer"; // String
```

```
let precoProduto = 850.50; // Number
```

```
const emEstoque = true; // Boolean
```

```
const categorias = ["Móveis", "Escritório", "Gamer"]; // Array
```

```
const especificacoes = { cor: "Preta", peso: 15 }; // Object
```

```
// Entendendo o Escopo e Mutabilidade
```

```
if (emEstoque) {
```

```
    let desconto = 50; // Variável de bloco
```

```
    var taxa = 10; // Variável "vazada"
```

```
    precoProduto = precoProduto - desconto; // let permite reatribuição
```

```
    // nomeProduto = "Mesa"; // ERRO! const não permite reatribuição
```

```
    console.log(`Preço com desconto: ${precoProduto}`);
```

```
}
```

```
// console.log(desconto); // ERRO! 'desconto' não existe fora do bloco if
```

```
console.log(taxa); // Funciona (imprime 10), mas é má prática pois 'var' vazou do bloco!
```

### E. Explicação linha por linha:

- **Linhas 2-6:** Inicializamos nossos dados simulando um produto de e-commerce. Usamos `const` para o que não muda (nome, categorias) e `let` para o que pode mudar (preço).
- **Linha 9:** Verificamos se está em estoque (um valor booleano puro).
- **Linha 10-11:** Criamos `desconto` com `let` (seguro, vive só no `if`) e `taxa` com `var` (inseguro).
- **Linha 12:** Atualizamos o valor de `precoProduto`. Como foi declarado com `let`, isso é perfeitamente válido.
- **Linha 13:** Comentada porque se tentarmos mudar `nomeProduto` (que é `const`), o JS lança um erro parando o programa.
- **Linha 17-18:** Tentamos acessar as variáveis fora do bloco `if`. O `desconto` dá erro (o que é correto e esperado), mas a `taxa` existe, provando o perigo do `var`.

## Manipulando informações: Operadores

### A. Conceito:

Para criar lógica, precisamos comparar e manipular nossos dados utilizando **operadores**.

- Temos os **aritméticos** (`+`, `-`, `*`, `/`, `%`) para cálculos matemáticos.
- Temos os de **atribuição** (`=`, `+=`, `-=`) para guardar valores nas variáveis.

- Os operadores de **comparação** (>, <, >=, <=, ===, !==) servem para testar valores e sempre nos retornam um Boolean (true ou false).

Uma característica vital do JavaScript é a comparação estrita. Nós sempre utilizamos três sinais de igual (===) em vez de dois (==). O operador == compara apenas o valor, fazendo coerções de tipo perigosas (por exemplo, transformar a string "5" no número 5 escondido). O operador === compara o valor e o tipo do dado, garantindo que "5" === 5 seja false, evitando bugs lógicos severos.

Por fim, utilizamos operadores **lógicos** (&& para E/AND, || para OU/OR, ! para NÃO/NOT) para combinar múltiplas comparações em uma única instrução de decisão. Com eles, nós conseguimos estruturar validações complexas, como checar se um campo de formulário não está vazio E se tem a quantidade mínima de caracteres.

## B. Explicação didática:

Pensem nos operadores como os juízes e os trabalhadores de uma fábrica.

- Os **aritméticos** são os trabalhadores de força bruta: eles cortam, somam, dividem os materiais.
- Os de **comparação** são os juízes na esteira de qualidade: eles olham para duas peças e dizem "Sim, são iguais" ou "Não, essa é maior que aquela".
- Os **lógicos** são os gerentes: eles dizem "A peça só passa se for da cor vermelha E (&&) tiver mais de 2kg".

Sobre o === vs ==: O == é aquele inspetor míope. Ele vê uma maçã de plástico e uma maçã real e diz que são iguais porque têm o mesmo formato. O === é o inspetor rigoroso: ele vê que o formato é igual, mas o material (o tipo do dado) é diferente, então ele reprova!

## C. Visualizando:

Um fluxograma simples onde duas caixas de entrada contendo o texto "10" e o número 10 passam por um funil chamado == saindo uma luz verde (true), e passam por um funil === saindo uma luz vermelha indicando false com o aviso "Tipos diferentes detectados!". {ver img02.png}

## D. Exemplo de Código:

JavaScript

```
let quantidadeCarrinho = 3;  
const precoUnitario = 150.00;
```

// Operadores aritméticos e de atribuição combinada

```
let total = quantidadeCarrinho * precoUnitario;  
total += 20; // Adiciona o frete (total = total + 20)
```

// Operadores de comparação e lógicos

```
const limiteEstoque = 10;  
const clienteVIP = true;
```

// O cliente pode comprar SE: pedir menos ou igual ao estoque E for VIP OU o valor for baixo

```
const compraAprovada = (quantidadeCarrinho <= limiteEstoque) && (clienteVIP || total < 100);
```

// Demonstração da comparação estrita

```
const idRecebido = "500";  
const idNoBanco = 500;  
const ehOMesmoProduto = (idRecebido === idNoBanco); // Retorna false!
```

```
console.log(`Compra Aprovada? ${compraAprovada}`);  
console.log(`Ids são idênticos? ${ehOMesmoProduto}`);
```

## E. Explicação linha por linha:

- **Linhas 5-6:** O operador `*` multiplica os valores. Na linha 6, usamos o operador de atribuição `+=` como um atalho elegante para somar o frete ao valor que já estava em `total`.
- **Linha 13:** Combinamos operadores lógicos. Os parênteses dão precedência. Avaliamos primeiro se o cliente é VIP ou se o total é baixo (`||`). Depois usamos o `&&` para garantir que a primeira condição (ter estoque) seja obrigatoriamente verdadeira.
- **Linhas 16-18:** Mostramos a diferença de tipos. Apesar do conteúdo legível ser 500, um é texto, o outro é número. Usando `===`, nos protegemos de cruzar dados errados, retornando `false`.

## Reutilizando a Lógica: Funções Utilitárias (utils.js)

### A. Conceito:

Funções são blocos de código encapsulados que executam uma tarefa específica. Em sistemas reais, nós evitamos escrever a mesma lógica de cálculo ou validação várias vezes pelo código. Ao invés disso, nós extraímos essa lógica para funções dedicadas. Quando agrupamos funções que resolvem problemas gerais (como formatar moeda, validar um e-mail, gerar uma data), nós criamos bibliotecas utilitárias, frequentemente armazenadas em um arquivo chamado `utils.js`.

O arquivo `utils.js` (abreviação de *utilities* ou utilitários) é um padrão de projeto extremamente comum em desenvolvimento de software, especialmente no JavaScript. Ele não é uma funcionalidade nativa da linguagem, mas sim uma **convenção arquitetural**.

Basicamente, o `utils.js` serve como uma "caixa de ferramentas" onde você agrupa funções genéricas, independentes e reutilizáveis que são úteis em várias partes do seu projeto, mas que não pertencem a nenhuma lógica de negócio específica.

#### 1. O que costuma ter dentro de um `utils.js`?

As funções aqui geralmente seguem o princípio de **"funções puras"** (o mesmo input sempre gera o mesmo output, sem efeitos colaterais). Exemplos comuns incluem:

- **Formatação:** Funções para formatar datas (ex: `formatarData(data)`), moedas (ex: `formatarMoeda(valor)`) ou strings (ex: `capitalize(texto)`).
- **Validação:** Verificações de formato de e-mail, CPF, ou checagem de tipos (ex: `isEmpty(obj)`).
- **Manipulação de Arrays/Objetos:** Funções auxiliares para buscar itens em listas complexas ou mesclar objetos.
- **Conversões:** Transformar unidades de medida, converter objetos para JSON, etc.

#### 2. Por que usar um arquivo de utilitários?

- **DRY (*Don't Repeat Yourself*):** Evita que você escreva a mesma lógica de formatação de data em dez arquivos diferentes.
- **Manutenibilidade:** Se a regra de formatação de um telefone mudar, você altera em **um único lugar** (`utils.js`) e todo o sistema é atualizado.
- **Organização:** Mantém a lógica de negócio principal limpa, retirando dela os "detalhes técnicos" repetitivos.

#### 3. Exemplo Prático

Imagine um projeto com um arquivo `utils.js`:

JavaScript

// utils.js

```
export const formatarMoeda = (valor) => {  
  return new Intl.NumberFormat('pt-BR', { style: 'currency', currency: 'BRL' }).format(valor);  
};
```

```
export const estaVazio = (valor) => {  
  return valor === null || valor === undefined || valor === "";  
};
```

Agora, em qualquer outro arquivo do projeto, você importa essas ferramentas:

JavaScript

// produto.js

```
import { formatarMoeda } from './utils.js';
```

```
const preco = 50;
```

```
console.log(formatarMoeda(preco)); // R$ 50,00
```

#### 4. Boas Práticas

Para que o `utils.js` não se torne um "lixão" de código desorganizado, siga estas dicas:

1. **Não coloque lógica de negócio:** Se a função só serve para o componente `Login`, ela não deve estar no `utils.js`. Ela pertence ao próprio componente ou a um serviço de autenticação.
2. **Mantenha pequenas:** Se o seu `utils.js` ficar grande demais, divida-o em arquivos temáticos (ex: `dateUtils.js`, `stringUtils.js`, `validationUtils.js`).
3. **Documente:** Como essas funções são genéricas, é vital que elas tenham comentários explicando o que recebem e o que devolvem.

#### B. Visualizando:

Um diagrama de blocos simples: À esquerda, caixas de entrada "Inputs" (ex: número `1500.5`). Setas apontam para uma caixa central chamada "Processamento: `formatarMoeda()`". Da caixa central, uma seta sai para a direita, chamada "Output / Return", contendo o valor `"R$ 1.500,50"`. {ver `img03.png`}

#### C. Exemplo de Código:

JavaScript

// Arquivo: `utils.js`

// Função para calcular o total do carrinho com desconto

```
function calcularTotal(preco, quantidade, descontoPercentual) {  
  if (quantidade <= 0) {  
    return 0; // Se não tem item, encerra e retorna 0 imediatamente  
  }  
  
  const subtotal = preco * quantidade;  
  const valorDesconto = subtotal * (descontoPercentual / 100);  
  const totalFinal = subtotal - valorDesconto;  
  
  return totalFinal; // Devolve o valor processado  
}
```

// Função para validar se um campo obrigatório foi preenchido corretamente

```
function validarCampoVazio(texto) {  
  // Retira espaços em branco das pontas e checa o tamanho  
  const textoLimpo = texto.trim();  
  
  if (textoLimpo.length === 0) {
```

```

    return false; // Campo está vazio ou só tem espaços
  }
  return true; // Campo é válido
}

// Testando nossas funções (isso seria feito em outro arquivo no projeto real)
const valorCarrinho = calcularTotal(200, 3, 10);
console.log(`O valor a pagar é: ${valorCarrinho}`);

const usuarioValido = validarCampoVazio(" "); // Apenas espaços
console.log(`O nome é válido? ${usuarioValido}`);

```

#### D. Explicação linha por linha:

- **Linhas 4-14:** Declaramos a função `calcularTotal` que espera 3 parâmetros.
- **Linhas 5-7:** O conceito de *Early Return* (retorno antecipado). Se a quantidade for inválida, a função devolve 0 e para a execução ali mesmo.
- **Linhas 9-11:** Executamos a regra de negócio matemática de forma clara e legível.
- **Linha 13:** O comando `return` pega a variável `totalFinal` e a "cospe" para fora da função.
- **Linhas 17-25:** Função que limpa os espaços invisíveis usando o método nativo `.trim()` e avalia o tamanho da string.
- **Linhas 28-32:** Simulamos o uso dessas funções. O retorno da função é armazenado nas constantes `valorCarrinho` e `usuarioValido` para uso posterior.

#### E. Dinâmica: A Arte do Early Return

**Objetivo:** Desenvolver o olhar crítico sobre legibilidade de código, carga cognitiva e manutenibilidade. Vocês devem entender que um código não serve apenas para o computador, mas principalmente para outros humanos (ou para o seu "eu" do futuro).

**Cenário:** Você recebeu um código legado de um sistema de e-commerce que calcula descontos. Ele funciona, mas é difícil de manter. Sua tarefa é analisar as duas propostas de implementação e decidir qual delas deve ser usada no projeto.

#### "Opção Pirâmide" (If/Else Aninhado)

*Foca em cercar o problema com camadas.*

JavaScript

```

function calcularDesconto(usuario, produto, cupom) {
  let desconto = 0;
  if (usuario.estaAtivo) {
    if (produto.quantidade > 0) {
      if (cupom.isValid) {
        desconto = 0.15;
      } else {
        desconto = 0.05;
      }
    } else {
      console.log("Erro: Produto esgotado");
      desconto = 0;
    }
  } else {

```



```
console.log("Erro: Usuário inativo");
desconto = 0;
}
return desconto;
}
```

## "Opção Direta" (Early Return / Guard Clauses)

Foca em limpar o caminho e sair rápido.

JavaScript

```
function calcularDesconto(usuario, produto, cupom) {
  // Cláusulas de Guarda: Saída antecipada
  if (!usuario.estaAtivo) return 0;
  if (produto.quantidade <= 0) return 0;

  // Lógica do Caminho Feliz
  if (cupom.isValid) return 0.15;

  return 0.05;
}
```

## Instruções para a Turma:

1. **Observação:** Leiam os dois blocos para discussão dentro das equipes.
2. **Debater para pensar:** Respondam às três perguntas abaixo e defendam a escolha técnica.
  - **Pergunta 1:** Qual código é mais fácil de explicar em voz alta para um colega?
  - **Pergunta 2:** Se precisarmos adicionar uma nova regra agora (ex: `if (usuario.isBlackFriday)`), qual código fica mais fácil de modificar sem quebrar o que já existe?
  - **Pergunta 3:** Por que a "Opção Pirâmide" pode ser considerada perigosa em projetos grandes?
3. **Entrega:** O Relatório de Refatoração.  
Cada equipe deve entregar um pequeno texto contendo:
  - **A Escolha Técnica:** Declarar qual opção (A ou B) será adotada como padrão no projeto.
  - **Justificativa "Clean Code"** (Mínimo 3 pontos): Listar três razões técnicas para a escolha.
    - **Exemplo:** "A Opção B reduz a profundidade do código (indentação)."
  - **O Teste de Stress:** Descrever em um parágrafo o que acontece no código escolhido se uma nova condição for adicionada: "E se o usuário for VIP e ganhar 20% de desconto?".

---

## FAQ - Dúvidas Reais

1. **Por que nós sempre usamos `===` em vez de `==` em JavaScript?**

**Resposta:** Porque o JavaScript tenta "ajudar" convertendo tipos secretamente se você usar `==`. A string `"0"` vira o número `0`. O `===` obriga que o valor E o tipo sejam idênticos, impedindo bugs silenciosos no sistema.

## 2. Posso voltar a usar `var` se eu preferir?

**Resposta:** Profissionalmente, não. O `var` sofre de vazamento de escopo e de *hoisting* confuso. O mercado adotou `let` e `const` para garantir que as variáveis existam apenas onde deveriam existir (dentro do seu bloco).

## 3. Qual a diferença entre `undefined` e `null`?

**Resposta:** `undefined` significa que a variável foi criada, mas o próprio JavaScript diz "não faço ideia do que tem aqui ainda". `null` é um valor proposital que o desenvolvedor atribui para dizer "esta variável está intencionalmente vazia no momento".

## 4. Se eu criar um `const` Array ou Object, não posso mais mexer nele?

**Resposta:** Você não pode reatribuir a variável inteira (ex: `meuArray = []`), mas você **pode** modificar o conteúdo interno dela, adicionando e removendo itens (`meuArray.push('novo item')`). O `const` bloqueia a referência de memória, não os elementos filhos.

## 5. Se a função não tiver a palavra `return`, o que acontece com o resultado?

**Resposta:** Ele se perde! A função fará os cálculos internamente, mas na hora de entregar a resposta para o sistema, ela entregará um valor `undefined`. É o erro mais comum para quem está começando.

---

## Exercícios de Fixação

**Entrega Final:** Um único arquivo chamado `utils.js` contendo todas as funções desenvolvidas abaixo, com seus respectivos nomes documentados.

### 1. Minhas Primeiras Variáveis

**Descrição:** Declare 3 variáveis: `nomeProduto` (imutável), `quantidadeEstoque` (mutável) e `produtoAtivo` (imutável). Atribua valores lógicos a elas correspondentes aos seus tipos adequados.

### 2. Saudação ao Cliente

**Descrição:** Crie uma função chamada `saudarCliente(nome)` que recebe uma string e retorna a frase: "Olá, [nome]! Bem-vindo à nossa loja."

### 3. Formatar para Moeda

**Descrição:** Crie uma função chamada `formatarMoedaBRL(valor)` que recebe um número (ex: 15.5) e retorna ele como string usando o padrão brasileiro (ex: "R\$ 15.50" - dica: pesquise sobre o método `.toFixed()`).

### 4. Desconto de Funcionário

**Descrição:** Crie uma função `calcularDesconto(precoOriginal, isFuncionario)`. Se o booleano for true, retorne o preço com 30% de desconto. Caso contrário, retorne o preço normal.

### 5. Montando o Produto

**Descrição:** Crie um objeto `produto` utilizando `const` que contenha as propriedades: `id`, `nome`, `preco` e `categorias` (um array de strings).

### 6. Validador de Senha Forte

**Descrição:** Crie uma função `validarSenha(senha)`. Ela deve retornar `true` apenas se a senha tiver 8 caracteres ou mais, e não puder ser a palavra exata "12345678" ou "senha".



## 7. Calculadora Total do Carrinho

**Descrição:** Crie uma função `fecharCarrinho(valorProduto, quantidade, valorFrete)`. Se o total dos produtos ( $\text{valorProduto} * \text{quantidade}$ ) for maior que 200, o frete vira 0. Retorne o valor total final da compra.

## 8. Validador de CPF (Tamanho)

**Descrição:** Crie uma função `validarTamanhoCPF(cpf)` que recebe uma string. Remova espaços em branco das pontas. Retorne `true` se a string resultante tiver exatamente 11 caracteres numéricos.

## 9. Checagem de Campo Obrigatório

**Descrição:** Modifique nossa função `validarCampoVazio` da aula. Agora, além de checar se está vazio, ela deve retornar `false` se o campo for `null` ou `undefined`.

## 10. Resumo da Compra

**Descrição:** Crie uma função `gerarResumo(nomeCliente, totalCompra)`. Essa função deve chamar internamente a sua função `formatarMoedaBRL` para formatar o total e retornar a string: "Cliente: [nome], Total a Pagar: [valor\_formatado]".

---

## Fechamento

### Conexão com a Situação de Aprendizagem:

As funções utilitárias que começamos a escrever hoje no `utils.js` (como validar se um campo está vazio ou calcular totais) não são apenas exercícios. Elas compõem o motor lógico do **frontend** da nossa **Situação de Aprendizagem**.

Antes do formulário de Login ou de Cadastro de Produtos bater lá no servidor PHP que construiremos depois, esse arquivo `utils.js` será responsável por blindar a interface, garantindo integridade de dados, segurança e usabilidade para o cliente final, conforme exigido pelos requisitos de Qualidade (QA) do nosso projeto.

### Na Próxima Aula:

Hoje aprendemos a criar a "inteligência" matemática e lógica do sistema. Mas como nós fazemos para capturar o que o usuário digitou na tela e jogar para dentro dessas funções? Na **Aula 16**, vamos abordar as **Condicionais** (`if/else`, `switch`) permitem que o código tome decisões e execute blocos específicos baseados em condições lógicas e os **Laços de repetição** (`for`, `while`) que automatizam a execução de uma tarefa várias vezes enquanto uma condição for verdadeira.